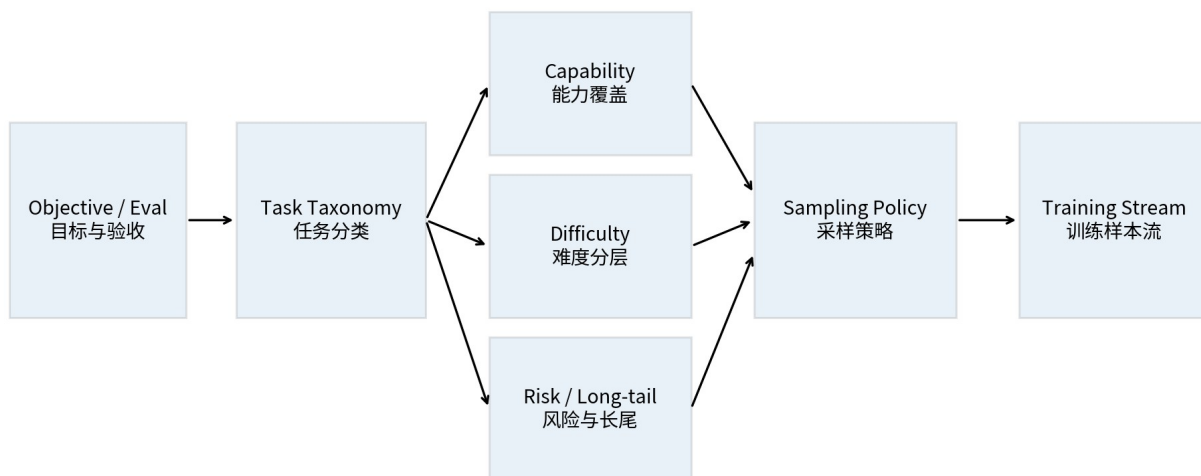


LLM 后训练之 Task Distribution

从 Task Taxonomy 到 Data Mixture / Sampling / Curriculum / Feedback Control

Deep Research & Engineering Handbook

Task Distribution Compiler：目标不是直接变成比例，而是先变成 taxonomy、覆盖、难度和约束



研究范围：2021-2026 公开论文、官方技术报告与开放后训练数据实践

研究日期：2026-08-13

核心结论 Task Distribution 不是“把若干数据集按百分比拼起来”，而是把 Objective/Eval 转化为训练 token 预算的控制策略：决定模型在什么能力、什么难度、什么输入分布、什么失败模式上花多少学习资源。

执行摘要

Task Distribution 是 LLM 后训练中连接“目标定义”和“数据工厂”的核心模块。它回答的不是“我们有什么数据”，而是“为了达到既定能力和行为目标，下一单位训练预算应该花在哪类任务上”。因此它本质上同时具有数据工程、优化与控制系统三种属性。

早期 instruction tuning 已经给出了两个稳定信号：其一，任务覆盖广度非常重要。FLAN/Flan Collection 发现扩大任务数量、平衡任务来源和混合 zero-shot/few-shot/CoT 形式显著影响未见任务泛化；Super-NaturalInstructions 在 1,616 个任务、76 种 task type 的框架下观察到，增加训练任务数量能持续改善跨任务泛化，而单任务实例数在其设置中大约到 64 条后已经趋于饱和。[1][2][3][4]

现代后训练把这一问题推进到了“能力定向混合”。Tulu 3 不直接按数据集大小拼接，而是先针对落后能力构造 skill-specific mixtures，训练模型寻找各能力的局部上界，再合并并通过 add/remove/downsample 反复修正落后能力。[7] DeepSeek-R1 则展示了另一种阶段化分布：reasoning-oriented RL、reasoning/non-reasoning SFT 与后续 mixed RL 使用不同的任务分布和奖励信号。[9]

进一步的研究正在从静态 heuristic 走向动态 mixture optimization：DoReMi 用 proxy model 学习域权重（虽然针对预训练，但方法论可迁移）；Dynamic Data Mixing、TAPIR、ADAPT 等工作分别探索随训练状态、任务难度或验证缺口动态调整采样比例。[10][11][12][14]

本手册的第一性模型 Task Distribution = Coverage × Balance × Difficulty × Quality × Risk × Stage × Feedback。真正工程化的对象不是一个固定比例表，而是一个 versioned sampling policy。

问题	错误理解	工程化理解
“数据集 A 30%，B 70%？”	只按样本数量配比	先定义能力/难度/风险目标，再把它映射到数据源
“哪个数据集最好？”	寻找单一万能数据集	估计每个数据源对不同能力的边际贡献与干扰
“更多数据一定更好？”	按规模堆数据	覆盖广度、信号质量与边际收益比原始规模更重要
“配一次就结束？”	静态 mixture	评测反馈驱动的动态重采样与 curriculum
“SFT/DPO/RL 用一套 prompts？”	共享同一分布	不同阶段承担不同学习目标，应有 stage-specific distributions

目录与阅读地图

章节	核心问题	关键产物
1. 定义	Task Distribution 到底是什么？	Distribution Contract
2. Taxonomy	如何把任务空间分解？	Task Taxonomy

章节	核心问题	关键产物
3. 广度 vs 深度	任务数与实例数怎么权衡?	Coverage Policy
4. Mixing	静态采样公式有哪些?	Sampling Policy
5. 难度	easy/hard 怎么配?	Difficulty Curriculum
6. 质量与冗余	高质量数据如何进入权重?	Quality-aware Mix
7. 长尾与风险	稀有但重要任务怎么处理?	Risk-weighted Mix
8. Stage-specific	SFT/DPO/RL 各自训什么?	Stage Distribution
9. Dynamic	如何自动调整比例?	Feedback Controller
10. Tulu 3 案例	开放后训练如何迭代配方?	Case Study
11. Reasoning/Agent	复杂任务如何分布?	Trajectory Mix
12. Eval Contract	如何证明配方有效?	Ablation & Eval
13. 失败模式	常见坑有哪些?	Failure Taxonomy
14. 工程模板	如何落地?	YAML / Dashboard / Checklist

1. Task Distribution 的第一性定义

给定后训练总预算 B (samples、tokens、GPU-hours 或 rollout 数)，Task Distribution 的任务是决定每个训练单元 i 获得多少预算 b_i ，使模型在目标评测向量上获得最大收益，同时满足安全、回归、成本和数据约束。它因此不是数据清单，而是一个资源分配问题。

给定任务集合 $T=\{t_1,\dots,t_K\}$ 和预算 B :

选择 sampling policy $p(t)$
使 Expected Utility(model_after_training) 最大

约束:

- $\sum p_i = 1$
- $p_i \geq \text{min_coverage}_i$
- $\text{risk_task exposure} \geq \text{floor}$
- $\text{regression} \leq \text{budget}$
- $\text{total_tokens} \leq B$

这里的“task”也不应该等同于“dataset”。一个 dataset 可能同时包含多种 task type、语言、难度与交互形态；一个能力也可能由多个 dataset 共同支持。工程上应建立 Task Node 与 Data Source 的 many-to-many 映射。

对象	含义	例子
Task Node	训练目标的最小可管理单元	coding / bug-localization / hard / multi-file
Data Source	样本来源	WildChat、FLAN、Persona synthetic、内部日志
Slice	对来源进行属性切片	Python + hard + tool-use + test-failure
Sampling Weight	该 slice 被抽到的概率	$p_i=0.035$
Budget	该 slice 最终消耗的 token/rollout	20M target tokens
Eval Link	该 slice 主要改善哪个指标	repo localization / SWE task success

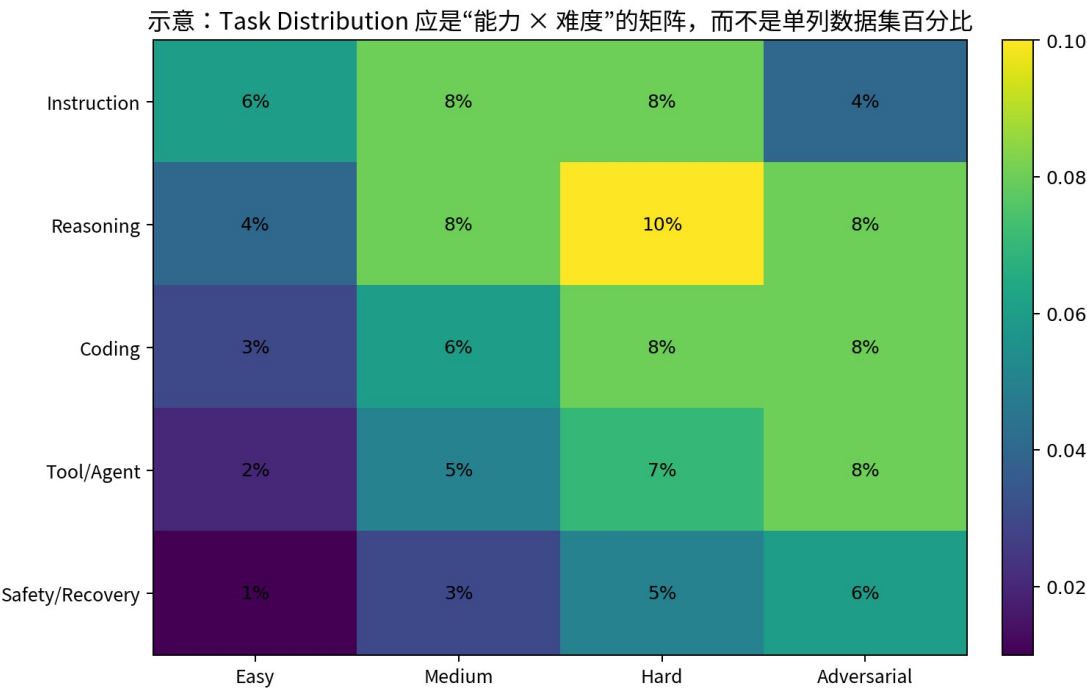
Know-Why 如果直接以 dataset 为采样单位，数据集规模、来源和任务能力会被绑死；团队会很难回答“为什么这个数据要占 12%”。Task Taxonomy 是 Distribution 的前置条件。

2. Task Taxonomy：先定义空间，再讨论比例

Super-NaturalInstructions 的重要贡献之一不是单纯“任务很多”，而是把任务沿 task type、language、domain 等维度组织起来；其 1,616 个任务覆盖 76 种 task type、55 种语言和 33 个 domain。[4] 这类多轴 taxonomy 比“数据集名称列表”更适合分析泛化。

对现代通用后训练，更实用的是构造一个多维 Task Cube。至少建议保留：Capability、Difficulty、Domain、Interaction Form、Language/Locale、Context Length、Risk/Failure Type、Stage。

轴	典型取值	为什么需要
Capability	instruction / reasoning / coding / tool / writing / safety	对应模型要学会什么
Difficulty	easy / medium / hard / adversarial	防止 easy data 淹没高价值信号
Domain	general / science / finance / software / enterprise	控制知识与语境迁移
Interaction	single-turn / multi-turn / tool trajectory	控制行为模式
Language	zh / en / multilingual / code-switch	避免高资源语言统治
Context	short / medium / long	控制长上下文能力与成本
Risk/Failure	normal / jailbreak / recovery / ambiguous	训练关键长尾和恢复策略
Stage	SFT / preference / RLVR	同一 task 在不同阶段产生不同训练信号



工程推导 Task Distribution 更准确地说是一个高维 joint distribution：p(capability, difficulty, domain, interaction, language, risk, ...)，最终的“数据集百分比”只是它被具体数据源实现后的边缘分布。

3. 广度 vs 深度：为什么 Task Coverage 往往比单任务堆量更重要

Instruction tuning 的目标之一是学习“根据自然语言任务定义切换行为”的元能力，因此重复同一任务上的大量实例，与增加新的任务映射并不等价。Super-NaturalInstructions 的受控实验显示：训练任务数量增加时，未见任务表现近似对数线性增长；而每个训练任务的实例数增加到约 64 后，其跨任务泛化已经基本饱和，大量实例反而增加训练时间和过拟合风险。[4]

Flan 2022 也观察到 held-out 性能随着任务数量扩展持续提高，并将 1,800+ tasks、task source balancing、input inversion 和混合 prompt settings 视为关键设计因素。[2][3] Scaling Instruction-Finetuned Language Models 报告 Flan-PaLM 540B 在约 1.8K tasks 上 instruction tuning 后，对多类任务平均大幅优于 base PaLM。[2]

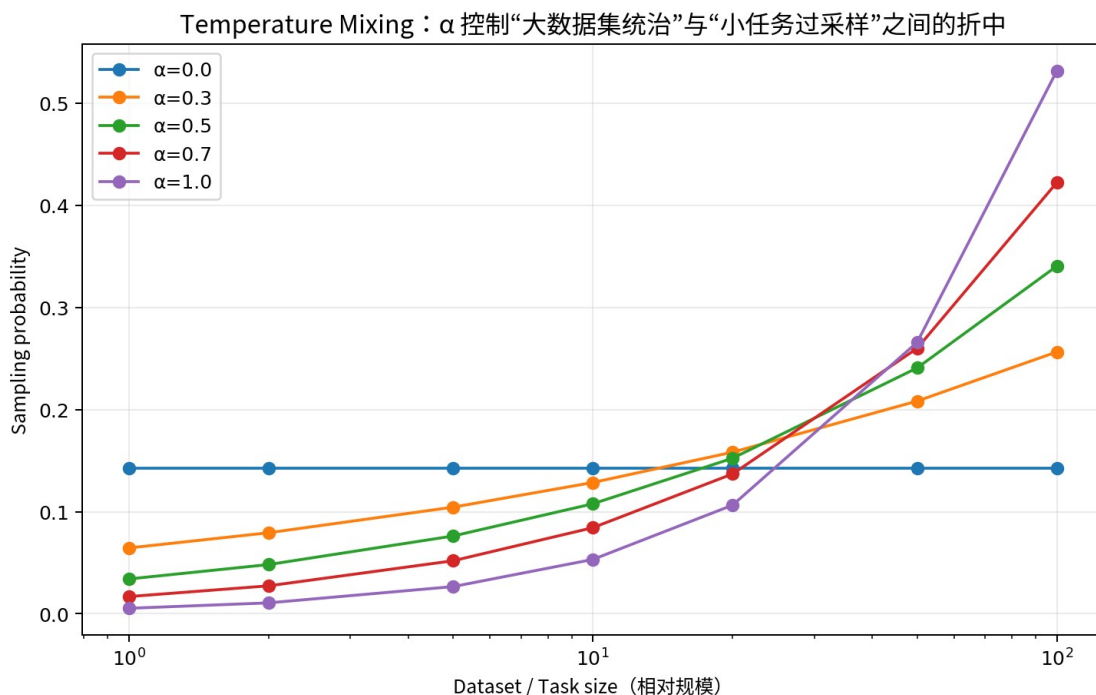
预算增加方式	更可能提高什么	主要风险	适用时机
增加新 Task Type	跨任务泛化、instruction following	新任务质量不稳定	能力覆盖仍有明显空洞
增加同 Task 实例	单任务拟合、边界覆盖	过拟合、重复	任务内部多样性仍不足
增加 Prompt Template	对措辞/格式鲁棒性	模板伪多样性	instruction sensitivity 高
增加 Difficulty	上限与推理能力	训练不稳定	easy 已饱和
增加 Failure/Negative	边界、恢复、拒绝/纠错	可能伤害正常分布	特定失败率高

Practical Rule 小预算阶段优先做 coverage audit；只有在某一能力节点已覆盖但方差/边界不足时，才继续增加该任务的实例深度。不要让百万级大数据集因为“手头有”而自动取得百万级权重。

4. 静态 Sampling：五种常见配方与权衡

Task Distribution 最终要落到 sampling probability。最基本的五类静态策略如下；没有一种在所有场景都最优，关键是明确你在保护什么。

策略	公式/直觉	优点	问题
Uniform over tasks	$p_i = 1/K$	最大化任务覆盖	小任务可能被严重过采样
Size-proportional	$p_i \propto n_i$	接近原始数据自然比例	大数据集统治训练
Temperature	$p_i \propto n_i^\alpha, 0 < \alpha < 1$	平滑大/小数据差距	α 仍需实验
Capped proportional	$p_i \propto \min(n_i, c)^\alpha$	限制巨型数据集权重	cap 需要与目标绑定
Priority weighted	$p_i \propto size_i^\alpha \times priority_i \times quality_i$	能编码产品目标	容易引入人为偏见



FLAN 2021 的公开训练细节体现了 capped proportional 思想：它限制每个数据集的训练样本规模，并采用 examples-proportional mixing，同时限制数据规模超过一定阈值后不再获得额外 sampling weight。[1] 这类做法的核心不是某个具体 cap 数字，而是防止原始 dataset size 被误当成“任务重要性”。

```
# 一个更可解释的静态权重模板
raw_i = (size_i ** alpha) * capability_priority_i * quality_i
raw_i *= difficulty_multiplier_i
raw_i *= risk_multiplier_i

p_i = raw_i / sum(raw)

# 再施加 floor / cap
p_i = project_to_constraints(p_i, min_coverage, max_share)
```

5. Difficulty Distribution：不是“越难越好”

难度是 Task Distribution 中最容易被忽略的轴。全部 easy 会让 loss 很好看但能力上限停滞；全部 hard 则会造成低成功率、低质量梯度或模型学会表面模式。更有效的策略通常是保证基础覆盖，然后逐步把预算转向当前模型“可学但尚未掌握”的区域。

难度区间	模型表现	训练价值	策略
Too Easy	几乎全对	边际收益低	降采样，仅保留行为/格式锚点
Learnable	有稳定错误但可被示范/反馈纠正	最高	重点采样

难度区间	模型表现	训练价值	策略
Frontier	成功率较低但存在正确 rollouts	高，但昂贵	用于 reasoning/RL/search
Too Hard	几乎无有效信号	低/不稳定	先分解任务或提高 teacher/verifier
Adversarial / Risk	自然频率低但风险高	高风险价值	按风险而非自然频率过采样

TAPIR 代表一种 task-aware curriculum 思路：根据 student 当前能力选择更难跟随的指令，并调整 task distribution 与难度日程；其目标是用更少数据获得更均衡的能力。[12] 2025 年的 ADAPT 则把固定配比进一步改写为预算约束下学习 task sampling proportions 的 meta-learning 问题。[14]

Know-How 难度不要只依赖题目文本的静态标签。更有效的 difficulty signal 是 model-relative：当前模型的 pass rate、reward margin、judge uncertainty、retry 次数、tool errors、teacher-student gap。难度会随训练阶段变化。

6. Quality、Diversity 与 Redundancy：Task 重不能只看量

LIMA 用仅 1,000 条精心筛选的 SFT 样本展示了 instruction alignment 对数据质量高度敏感。[6] 这并不意味着“大数据无用”，而是说明同一训练预算下，不同样本的边际价值差异很大。Task Distribution 因此需要把 quality 与 diversity 作为 sampling weight 的组成部分，而不是只在前置 filtering 做一次二元删选。

信号	可以测什么	如何进入 Distribution
Correctness	答案/trajectory 是否正确	低正确率 source 降权或只做 negative
Instruction quality	prompt 是否清晰、有意义	低质量 instruction 降权
Response quality	teacher response 是否完整准确	quality score 作为 weight
Novelty	与已训练样本距离	重复簇降采样
Skill uniqueness	是否补充缺失能力	稀缺 skill 加权
Gradient / loss	模型是否仍能从中学习	低 loss 饱和任务逐步降权
Eval transfer	该数据对 held-out eval 的增益	作为最终边际收益依据

2025 年 “Fixing It in Post” 对 Tulu-3-SFT-Mix 与 SmolTalk 做样本级质量/任务结构分析，并构造出样本数减少约 14% 但在关键 benchmark 上匹配或超过源数据集的 TuluTalk mixture。[13] 这类结果强化了一个工程原则：mixture optimization 的目标是 information gain per token，而不是 retained rows。

分布与过滤的区别 Filtering 回答“这个样本能不能进池子”；Distribution 回答“即使都合格，谁应该被看 10 次、谁只看 1 次”。二者不能合并成一个 quality threshold。

7. Long-tail / Risk Distribution：自然频率不是训练频率

真实线上分布中，高影响失败往往是低频的：tool failure、ambiguous request、long-context conflict、jailbreak、wrong completion claim、multi-step recovery 等。如果完全按自然频率采样，这些任务几乎无法形成稳定梯度。

Task 类型	线上频率	风险/价值	训练策略
普通问答	高	中	接近目标流量或适度下采样
复杂推理	中/低	高能力价值	显著过采样
工具失败恢复	低	高 Agent 价值	构造 synthetic failure 并过采样
高风险边界	极低	高安全价值	设置 minimum exposure floor
罕见语言/locale	低	公平/可用性价值	按目标市场设 floor
回归保护任务	低	保护旧能力	每轮固定 replay quota

```
# 风险加权不是概率估计，而是决策价值
training_weight_i ∝
  online_frequency_i
  * business_impact_i
  * failure_cost_i
  * learnability_i

# 对 hard constraints 通常还需要 minimum quota
p_i >= risk_floor_i
```

关键区分 Target deployment distribution 与 Training distribution 不应被强制相等。前者决定最终评测要覆盖什么；后者可以故意过采样低频、高价值、可学习的任务，以更有效地改变模型。

8. Stage-specific Distribution：SFT / Preference / RL 不应共享同一配方

现代后训练是多阶段的，每个阶段需要的监督信号不同，因此 Task Distribution 也应该分阶段设计。Tulu 3 的公开 recipe 就把 prompt pool 作为所有阶段的基础，但在 SFT、preference tuning 和 RLVR 中选择不同子集和数据构造方法。[7]

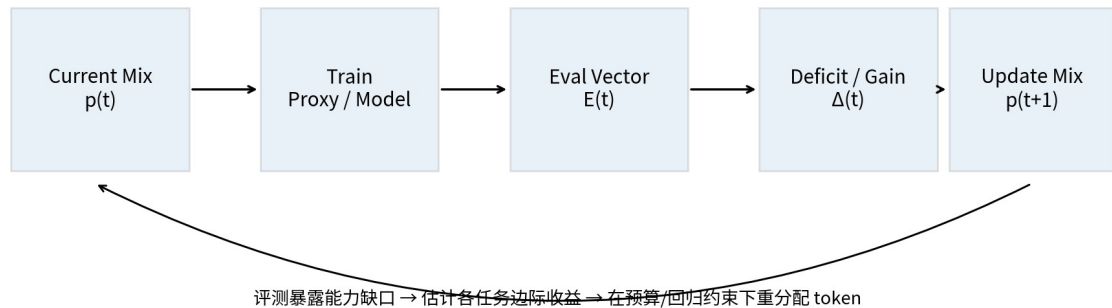
阶段	最主要目标	更适合的任务分布	不应过度依赖
SFT	建立 behavior prior / skills	覆盖广、答案高质量、任务多样	大量近重复 easy samples
Preference / DPO	学习细粒度 better/worse	有可比较候选、风格/指令/安全边界	无法区分优劣的 pair
RLVR	探索可验证更优策略	数学/代码/可执行环境等 verifier tasks	纯主观且 reward 不稳定任务
General RL / RM	偏好与行为约束	真实用户分布 + 风险 slices	单一 reward proxy
Distillation	压缩 teacher 已发现行为	teacher 强项 + student 缺口	teacher 的全部自然输出

DeepSeek-R1 是 stage distribution 的强案例：R1-Zero 的 RL 窄聚焦于 reasoning，导致写作与 open-domain QA 能力有限；R1 随后加入 cold-start、reasoning + non-reasoning SFT，并在第二阶段 RL 中混合 reasoning-focused 与 general-purpose data。Nature 版本还明确说明 general instruction data 与 preference-based rewards 只在第二 RL 阶段的最后一部分训练步骤中引入。[9]

Know-Why 同一份 prompt 在 SFT 中可能提供“模仿信号”，在 DPO 中提供“排序信号”，在 RL 中提供“探索空间”。因此 Stage Distribution 不是复制同一百分比，而是围绕每种 objective 选择最有信息量的任务。

9. Dynamic Task Distribution：从静态配比升级为反馈控制

Dynamic Task Distribution = Feedback Control，而不是一次性 Excel 配比



静态 mix 的最大问题是：模型能力在训练中不断变化，但 sampling weight 不变。一个任务从“高价值 learnable”变成“已饱和”后继续占据大量预算，就是浪费；另一个原本太难的任务可能随着基础能力提升变成最有价值的 frontier task。

因此更成熟的系统会周期性计算每个 task/slice 的 deficit、learning progress、quality、coverage 与 interference，再更新权重。DoReMi 在预训练场景中小 proxy model + group DRO 学习域权重，并显示 proxy 学到的 mixture 可以迁移到大模型；这个“先低成本估权，再大规模训练”的思想对 post-training 也非常有价值。[10]

Dynamic Data Mixing 进一步直接针对 instruction tuning，指出 fixed task weights 无法反映模型训练状态变化，并尝试依据 dataset redundancy 动态调整采样。[11] ADAPT 则将 budget-constrained instruction tuning 的 task proportions 作为可学习变量，围绕 worst-case validation objective 做自适应 curriculum。[14]

```
# 一个工程可落地的动态更新思路（示意）
score_i(t) =
    w1 * eval_deficit_i(t)
  + w2 * learning_progress_i(t)
  + w3 * risk_priority_i
  + w4 * novelty_i
  - w5 * redundancy_i
  - w6 * regression_interference_i

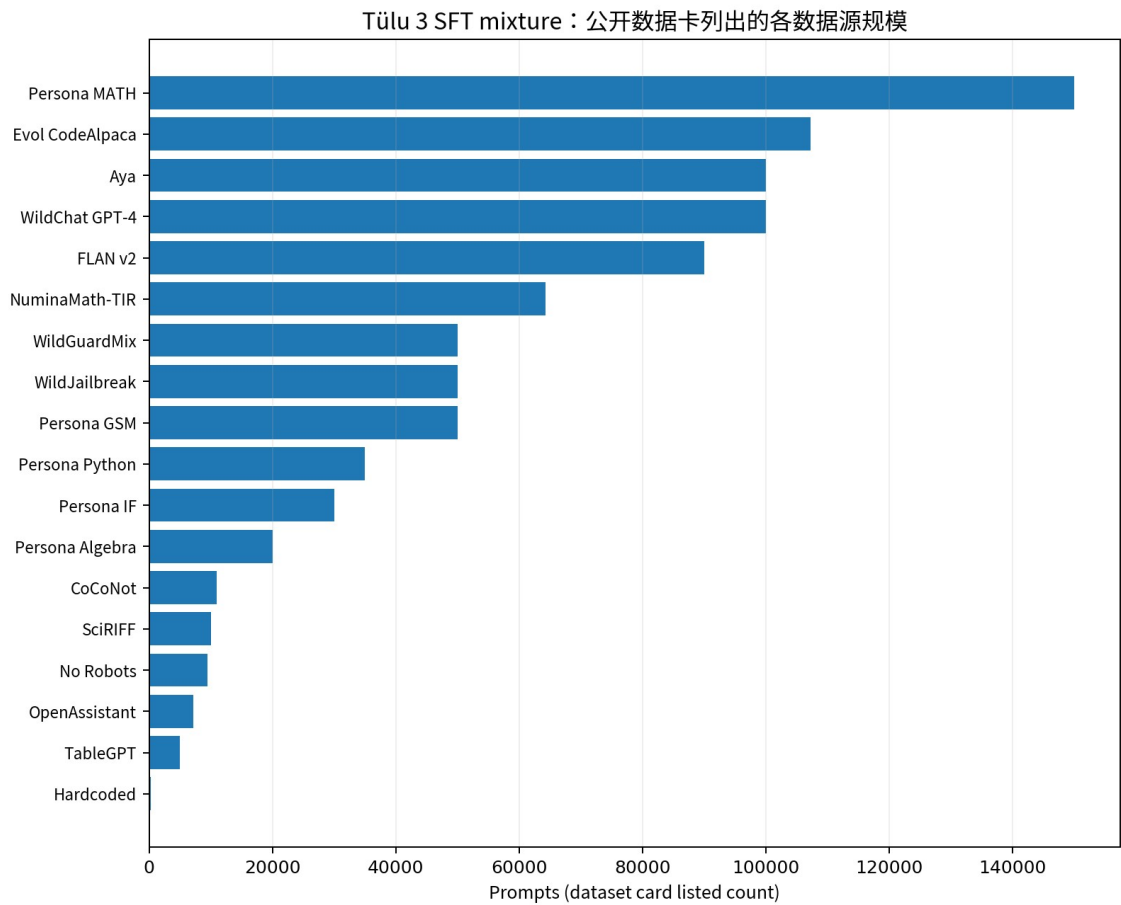
p_i(t+1) = softmax(score_i / temperature)
project p onto min/max quota constraints
```

不要直接追逐 loss 训练 loss 高可能代表“很值得学”，也可能代表数据错误、grader 不一致或任务超出模型能力。动态分布必须同时读取 eval gain、quality 和 learnability，而不是只用 loss 排序。

10. Tulu 3：开放后训练中的 Task Distribution 案例

Tulu 3 是目前最有参考价值的开放 post-training recipe 之一。其公开方法不是先确定一个固定百分比，而是：先用基线模型识别落后能力；为每项能力分别收集/合成高质量数据并训练 skill-specific mixtures；保留单项能力表现最好的 mixture；再把它们合并成 preview mix，之后继续通过添加/删除数据集、downsample 超大数据集和 decontamination 来修正落后能力。[7]

Ai2 官方博客还披露：用于后续 recipe 的 prompt pool 约 939,344 条，其中约 57% 来自公开资源、43% 为内部合成；合成 prompt 被用于填补 precise instruction following、math、coding 等能力缺口。[7]



值得注意的是，当前公开数据卡在文档层面还提供了一个“数据治理反例”：页面声明 mixture 有 939,344 samples、viewer 显示 19 个 source 类别，但页面正文列出的 18 个 source count 相加为 889,344，存在 50,000 条的文档计数差异。[8] 这不影响 Tulu 3 方法论，但非常适合提醒工程团队：正式 Task Distribution 必须从可审计 manifest / dataset snapshot 计算，而不能把 README 文本当成唯一真相。

Tulu 3 做法	抽象成 Know-How
先找 lagging skills	从 Eval deficit 开始，而不是从可用数据开始
skill-specific mixture	先做局部上界实验，估计每类数据的能力贡献
再合并 global mixture	处理能力之间的干扰与协同
add/remove dataset	把 mixture 当作实验变量
downsample large datasets	数据规模不等于任务价值

Tulu 3 做法	抽象成 Know-How
synthetic fill gaps	缺失能力用目标导向生成补齐
decontamination	训练提升必须与评测泄漏分离

11. Reasoning / Coding / Agent 的 Task Distribution 有什么不同

系统	Distribution 的主单位	最重要轴	典型过采样对象
General Assistant	prompt / conversation	domain, intent, language, style	真实用户长尾、instruction constraints
Reasoning Model	problem / rollout	difficulty, verifiability, reasoning domain	frontier math/code/logical tasks
Coding Model	problem / code artifact	language, task type, testability	debug、repo-level、test failure
Tool Agent	trajectory	tool type, horizon, failure state	invalid args、tool fail、recovery
Research Agent	trajectory / source set	search depth, source quality, synthesis	contradictory evidence、long horizon

Reasoning/RL 系统尤其要区分 problem distribution 与 rollout distribution：问题被采样一次后，可能生成 N 条 trajectories。一个 task 占 10% prompts，不代表只占 10% compute；hard tasks 可能因为更长 CoT、更高 sample count 或 verifier cost 消耗更大预算。

```
# Agent / RL 更建议按 compute-normalized budget 管理
EffectiveBudget_i =
    num_prompts_i
    * rollouts_per_prompt_i
    * avg_tokens_per_rollout_i
    * verifier_cost_i

# 所以 dashboard 至少同时显示：
prompt_share, token_share, GPU_share, successful_signal_share
```

对于 Agent，训练分布还要包含 trajectory state，而不只是最终任务：正常执行、工具报错、部分成功、回滚、环境变化、上下文截断等。如果只训练“理想世界中的成功轨迹”，上线后模型缺乏 recovery policy。

12. Eval Contract：Task Distribution 必用消融明，而不是凭经验争论

Mixture 的效果是高度交互的：某数据源单独有效，加入全局 mix 后可能因为冗余或梯度干扰而无增益；某数据源看似对自身 benchmark 无帮助，却可能改善 instruction robustness。因此 Task Distribution 的核心实验单位应该是 controlled ablation，而不是只看最终模型。

实验	目的	建议做法
Leave-one-source-out	数据源是否真正有贡献	固定 tokens，移除一个 source 后补齐预算
Add-one-skill	新 skill 是否带来 transfer/干扰	从统一 baseline 添加固定 token
Weight sweep	寻找比例敏感区	0 / low / medium / high 四档
Budget-normalized	避免“更多 token”伪增益	所有 mix 保持相同有效 token
Proxy model	低成本探索 mixture	小模型筛配方，大模型验证
Held-out capability	测真正泛化	按 task category 做 held-out
Regression suite	检测能力冲突	同时观察旧能力与行为

FLAN、OPT-IML 和 Tulu 3 都采用了不同形式的 held-out/multi-task evaluation 来分析 mixture 决策。[3][5][7] 对生产团队而言，最重要的是把 Eval 指标按 Capability Spec 对齐，避免用一个平均分掩盖某个 P0 能力持续退化。

Distribution KPI 建议同时维护四类指标：Coverage（哪些目标 task 没数据）、Exposure（训练实际看了多少）、Marginal Gain（每百万 token 带来多少 eval 增益）、Interference（对其他能力造成多少回归）。

13. Task Distribution 的 14 类典型失败模式

失败模式	症状	根因	修复
1. Dataset-size dominance	最大数据集占大部分 token	把规模当重要性	temperature/cap + priority
2. Equal-source illusion	每数据集同权但 task 不同	dataset != task	先映射 taxonomy
3. Easy-data flooding	loss 好看，hard eval 不涨	easy 边际收益已低	difficulty rebalance
4. Benchmark overfit	某榜单飞涨、泛化差	分布贴 benchmark	held-out categories
5. Missing long tail	平均好、关键失败严重	按自然频率训练	risk floors
6. Static forever	后期大量预算浪费	模型状态变化	dynamic reweighting

失败模式	症状	根因	修复
7. Quality blind	高权重 source 有噪声	只看数量	quality-aware sampling
8. Diversity fake	模板多但语义重复	synthetic mode collapse	cluster / novelty audit
9. Stage sharing	SFT/DPO/RL 都同一 prompts	监督信号不同	stage-specific policy
10. Prompt-share only	RL 计算成本爆炸	忽略 rollout length	token/GPU normalized
11. No regression replay	新能力涨、旧能力掉	没有 replay quota	fixed regression mix
12. Eval contamination	配方 “异常有效”	训练-评测重合	strict decontamination
13. README as truth	账面比例对不上实际流	缺 data lineage	manifest + snapshot stats
14. No attribution	只知道最终模型变了	缺消融	mixture experiment matrix

14. 工程落地：Distribution Contract 与配置模板

建议把 Task Distribution 作为独立版本化 artifact，而不是散落在 dataloader 代码中。至少应包含 target taxonomy、source mapping、sampling formula、floors/caps、budget unit、stage、evaluation links 和 lineage。

```
version: task-dist-v2.4
stage: sft
budget_unit: target_tokens
total_budget: 1200000000

axes:
  capability: [instruction, reasoning, coding, tool_agent, safety]
  difficulty: [easy, medium, hard, adversarial]
  language: [zh, en, multilingual]

policy:
  base: temperature
  alpha: 0.45
  normalize_by: effective_tokens

constraints:
  max_source_share: 0.18
  min_capability_share:
    reasoning: 0.18
    coding: 0.15
    tool_agent: 0.10
    safety: 0.08
  replay_share: 0.10

dynamic_update:
  every_steps: 1000
  signals: [eval_deficit, learning_progress, quality, redundancy]
  max_weight_change_per_update: 0.20
```

14.1 一个完整的 Distribution Review 表

Review 问题	必须回答
目标映射	每个 P0 capability 是否至少有一个训练 slice 和一个独立 eval?
覆盖	是否存在目标 task 训练 exposure 接近 0?
规模偏置	最大 source 占 prompt/token/GPU 三种预算各多少?
难度	当前模型的 easy/learnable/frontier 比例是多少?
长尾	高风险低频任务有没有 minimum quota?
质量	高权重数据的错误率、重复率、teacher quality 如何?
阶段	当前阶段为什么需要这些 task, 而不是上一阶段的配方?
回归	是否保留旧能力 replay?
动态性	哪些指标会触发增权/降权?
可审计	实际训练 stream 能否重建 source/slice/weight/version?

15. Task Distribution 成熟度模型：“ 据” 到“控制”

等级	特征	团队状态
L0 Pile	把所有可用数据 concat	无法解释比例与能力关系
L1 Static Mix	人工设 dataset 百分比	可复现, 但依赖经验
L2 Taxonomy-driven	按 capability/difficulty/source 管理	目标可追踪到训练 slices
L3 Eval-driven	用消融与 deficit 迭代 mixture	配方是实验产物
L4 Dynamic Controller	按模型状态自动重采样, 并有约束/回归保护	Task Distribution 成为训练控制系统

最终判断标准 成熟团队不是“有一张漂亮的 20/30/50 配比表”，而是能够回答：为什么是这个比例？如果模型某能力突然落后，下一轮如何改变？如果数据源翻倍，为什么其权重不一定翻倍？如果训练预算减半，优先保留哪些 token？

考料究依据

来源	本手册使用位置
[1] Wei et al., Finetuned Language Models Are Zero-Shot Learners, ICLR 2022 / arXiv:2109.01652.	早期 FLAN：任务集混合、任务簇与 examples-proportional/capped mixing 思想。
[2] Chung et al., Scaling Instruction-Finetuned Language Models, JMLR 2024 / arXiv:2210.11416.	扩展任务数、模型规模、CoT 数据；Flan-PaLM 在约 1.8K tasks 上的结果。
[3] Longpre et al., The Flan Collection: Designing Data and Methods for Effective Instruction Tuning, ICML 2023 / arXiv:2301.13688.	task balancing、mixed prompting、task scaling 与 enrichment 消融。
[4] Wang et al., Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks, EMNLP 2022 / arXiv:2204.07705.	1,616 tasks、76 task types；任务数 vs 单任务实例数的 scaling analysis。
[5] Iyer et al., OPT-IML: Scaling Language Model Instruction Meta Learning through the Lens of Generalization, arXiv:2212.12017.	2,000 tasks；task sampling、specialized datasets 与三种 generalization 视角。
[6] Zhou et al., LIMA: Less Is More for Alignment, NeurIPS 2023 / arXiv:2305.11206.	高质量少量 SFT 数据对 alignment 的影响。
[7] Lambert et al., Tülu 3: Pushing Frontiers in Open Language Model Post-Training, arXiv:2411.15124；Ai2 Tülu 3 technical blog.	skill-specific mixture、迭代 add/remove/downsample、prompt curation。
[8] Ai2, allenai/tulu-3-sft-mixture dataset card, current public dataset snapshot.	公开 mixture source counts、939k row 规模与 provenance。
[9] Guo et al., DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning, Nature 645, 633-638 (2025); arXiv:2501.12948.	多阶段 reasoning/general task distribution 与 mixed rewards。
[10] Xie et al., DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining, NeurIPS 2023 / arXiv:2305.10429.	proxy model + Group DRO 学习 data mixture；用于方法论类比。
[11] Zhu et al., Dynamic Data Mixing Maximizes Instruction Tuning for Mixture-of-Experts, arXiv:2406.11256.	instruction tuning 的动态 task mixing 与 redundancy-aware weighting。
[12] Yue et al., Distilling Instruction-following Abilities of Large Language Models with Task-aware Curriculum Planning, arXiv:2405.13448.	task-aware distribution 与 curriculum difficulty。
[13] Djuhera et al., Fixing It in Post: A Comparative Study of LLM Post-Training Data Quality and Model Performance, arXiv:2506.06522.	Tulu-3/SmolTalk 分析与更精简的 TuluTalk mixture。
[14] Kadasi et al., ADAPT: Learning Task Mixtures for Budget-Constrained Instruction Tuning, arXiv:2512.04555.	在 token budget 下用 meta-learning 学习 task sampling proportions；较新的研究方向。
[15] Sanh et al., Multitask Prompted Training Enables Zero-Shot Task Generalization, ICLR 2022 / arXiv:2110.08207.	T0：显式 multitask prompted training 与 held-out task generalization。